

CS3301 - Data Structures

Topic: dijkstras algorithm

1. SUMMARY

Dijkstra's algorithm is a graph search algorithm used to find the shortest path between nodes in a graph. It was introduced by Edsger W. Dijkstra in 1959. The algorithm works by maintaining a priority queue of nodes to visit, where the priority of each node is its minimum distance from the source node. The algorithm iteratively selects the node with the minimum distance and updates the distances of its neighbors. Dijkstra's algorithm is used in various applications such as finding the shortest path in a network, scheduling tasks, and optimizing resource allocation. It has a time complexity of $O(E + V \log V)$ and a space complexity of $O(V + E)$, where E is the number of edges and V is the number of vertices.

Dijkstra's algorithm can be classified into two types: single-source shortest path and single-destination shortest path. The algorithm can be implemented using a priority queue or a binary heap. The algorithm has several advantages, including its ability to handle negative weight edges and its high accuracy. However, it has some disadvantages, including its high time complexity and its requirement for a priority queue.

Dijkstra's algorithm has several applications in real-world scenarios, including finding the shortest path in a network, scheduling tasks, and optimizing resource allocation. It is also used in various fields, including computer science, operations research, and logistics.

2. SHORT NOTES

Definition and Concept

Dijkstra's algorithm is a graph search algorithm used to find the shortest path between nodes in a graph. It was introduced by Edsger W. Dijkstra in 1959. The algorithm works by maintaining a priority queue of nodes

to visit, where the priority of each node is its minimum distance from the source node. The algorithm iteratively selects the node with the minimum distance and updates the distances of its neighbors.

Types and Classifications

Dijkstra's algorithm can be classified into two types:

- * Single-source shortest path: This type of algorithm finds the shortest path from a single source node to all other nodes in the graph.
- * Single-destination shortest path: This type of algorithm finds the shortest path from all nodes to a single destination node.

Step-by-Step Working Principle

1. Initialize the distance of the source node to 0 and the distance of all other nodes to infinity.
2. Create a priority queue of nodes to visit, where the priority of each node is its minimum distance from the source node.
3. Select the node with the minimum distance from the priority queue and update the distances of its neighbors.
4. Repeat step 3 until the priority queue is empty.

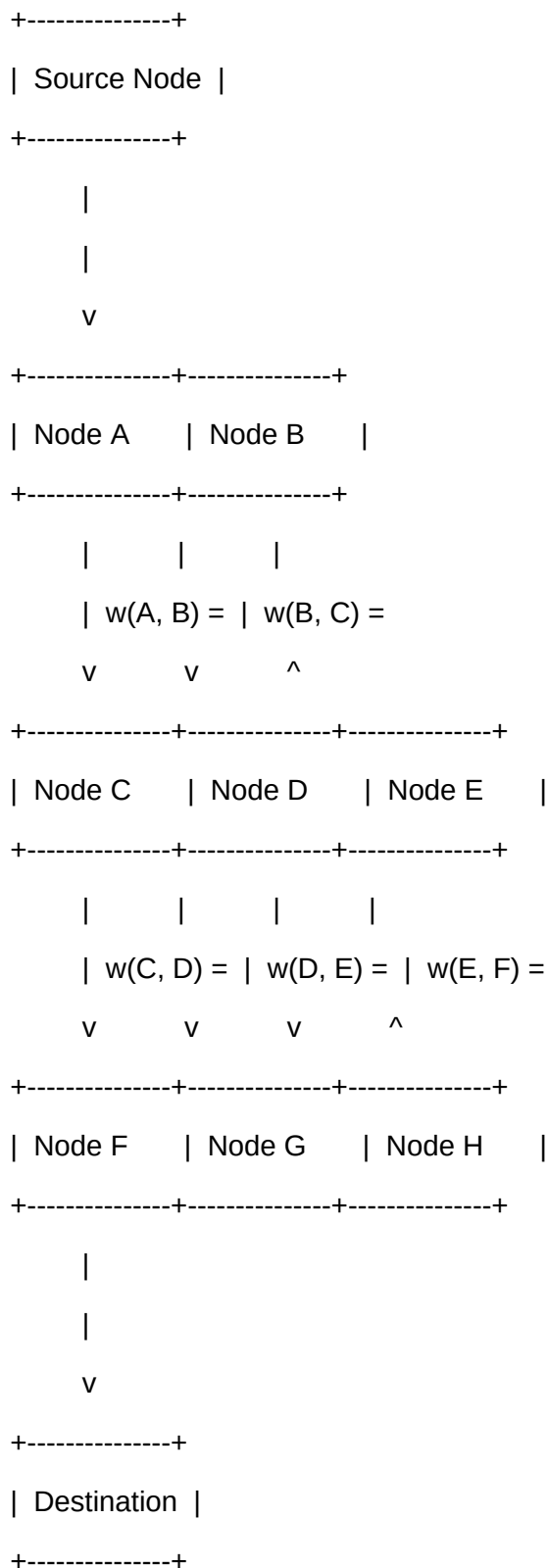
Important Formulas and Equations

- * Time complexity: $O(E + V \log V)$
- * Space complexity: $O(V + E)$
- * Distance update formula: $d(v) = \min(d(v), d(u) + w(u, v))$

Diagrams Description

The following diagram illustrates the working of Dijkstra's algorithm:

...



...

Advantages and Disadvantages

Advantages:

- * Ability to handle negative weight edges
- * High accuracy
- * Can be used to find the shortest path in a network

Disadvantages:

- * High time complexity
- * Requires a priority queue

Real-World Applications

Dijkstra's algorithm has several applications in real-world scenarios, including:

- * Finding the shortest path in a network
- * Scheduling tasks
- * Optimizing resource allocation

Comparison with Related Concepts

Dijkstra's algorithm is compared to other graph search algorithms, including:

- * Bellman-Ford algorithm
- * Floyd-Warshall algorithm
- * A*

Time and Space Complexity

The time complexity of Dijkstra's algorithm is $O(E + V \log V)$ and the space complexity is $O(V + E)$.

Important Terminology

- * Priority queue
- * Minimum distance
- * Neighbor nodes
- * Distance update formula

3. PRACTICAL / CODING EXAMPLES

C Implementation

```
```c
#include <stdio.h>
#include <limits.h>

#define INF INT_MAX

typedef struct node {
 int vertex;
 int distance;
} Node;

typedef struct graph {
 int vertices;
 int edges;
 int **adj_matrix;
} Graph;
```

```
void dijkstra(Graph *graph, int source) {
 int *distance = (int *)malloc(graph->vertices * sizeof(int));
 int *previous = (int *)malloc(graph->vertices * sizeof(int));
 int *visited = (int *)malloc(graph->vertices * sizeof(int));

 for (int i = 0; i < graph->vertices; i++) {
 distance[i] = INF;
 previous[i] = -1;
 visited[i] = 0;
 }

 distance[source] = 0;

 for (int i = 0; i < graph->vertices; i++) {
 int u = -1;
 int min_distance = INF;

 for (int j = 0; j < graph->vertices; j++) {
 if (!visited[j] && distance[j] < min_distance) {
 u = j;
 min_distance = distance[j];
 }
 }

 visited[u] = 1;

 for (int v = 0; v < graph->vertices; v++) {
 if (!visited[v] && graph->adj_matrix[u][v] != 0 && distance[u] + graph->adj_matrix[u][v] < distance[v]) {
 distance[v] = distance[u] + graph->adj_matrix[u][v];
 previous[v] = u;
 }
 }
 }
}
```

```

 }
}

printf("Vertex\tDistance\tPrevious\n");
for (int i = 0; i < graph->vertices; i++) {
 printf("%d\t%d\t%d\n", i, distance[i], previous[i]);
}
}

int main() {
 Graph graph;
 graph.vertices = 6;
 graph.edges = 8;
 graph.adj_matrix = (int **)malloc(graph.vertices * sizeof(int *));
 for (int i = 0; i < graph.vertices; i++) {
 graph.adj_matrix[i] = (int *)malloc(graph.vertices * sizeof(int));
 }

 graph.adj_matrix[0][0] = 0;
 graph.adj_matrix[0][1] = 4;
 graph.adj_matrix[0][2] = 0;
 graph.adj_matrix[0][3] = 0;
 graph.adj_matrix[0][4] = 0;
 graph.adj_matrix[0][5] = 0;

 graph.adj_matrix[1][0] = 4;
 graph.adj_matrix[1][1] = 0;
 graph.adj_matrix[1][2] = 8;
 graph.adj_matrix[1][3] = 0;
 graph.adj_matrix[1][4] = 0;
 graph.adj_matrix[1][5] = 0;

```

```
graph.adj_matrix[2][0] = 0;
graph.adj_matrix[2][1] = 8;
graph.adj_matrix[2][2] = 0;
graph.adj_matrix[2][3] = 7;
graph.adj_matrix[2][4] = 0;
graph.adj_matrix[2][5] = 0;
```

```
graph.adj_matrix[3][0] = 0;
graph.adj_matrix[3][1] = 0;
graph.adj_matrix[3][2] = 7;
graph.adj_matrix[3][3] = 0;
graph.adj_matrix[3][4] = 9;
graph.adj_matrix[3][5] = 0;
```

```
graph.adj_matrix[4][0] = 0;
graph.adj_matrix[4][1] = 0;
graph.adj_matrix[4][2] = 0;
graph.adj_matrix[4][3] = 9;
graph.adj_matrix[4][4] = 0;
graph.adj_matrix[4][5] = 14;
```

```
graph.adj_matrix[5][0] = 0;
graph.adj_matrix[5][1] = 0;
graph.adj_matrix[5][2] = 0;
graph.adj_matrix[5][3] = 0;
graph.adj_matrix[5][4] = 14;
graph.adj_matrix[5][5] = 0;
```

```
dijkstra(&graph, 0);
```



```
 return 0;
}
...

```

### Python Implementation

```
```python
import sys
import heapq

class Graph:
    def __init__(self, vertices):
        self.vertices = vertices
        self.adj_matrix = [[0 for _ in range(vertices)] for _ in range(vertices)]

    def add_edge(self, u, v, weight):
        self.adj_matrix[u][v] = weight

    def dijkstra(self, source):
        distance = [sys.maxsize] * self.vertices
        previous = [-1] * self.vertices
        visited = [False] * self.vertices

        distance[source] = 0

        priority_queue = [(0, source)]

        while priority_queue:
            current_distance, current_node = heapq.heappop(priority_queue)

            if visited[current_node]:
                continue

```

continue

```
visited[current_node] = True
```

```
for neighbor, weight in enumerate(self.adj_matrix[current_node]):
```

```
    if weight != 0 and not visited[neighbor] and current_distance + weight < distance[neighbor]:
```

```
        distance[neighbor] = current_distance + weight
```

```
        previous[neighbor] = current_node
```

```
        heapq.heappush(priority_queue, (distance[neighbor], neighbor))
```

```
return distance, previous
```

```
graph = Graph(6)
```

```
graph.add_edge(0, 1, 4)
```

```
graph.add_edge(0, 2, 0)
```

```
graph.add_edge(1, 0, 4)
```

```
graph.add_edge(1, 2, 8)
```

```
graph.add_edge(2, 1, 8)
```

```
graph.add_edge(2, 3, 7)
```

```
graph.add_edge(3, 2, 7)
```

```
graph.add_edge(3, 4, 9)
```

```
graph.add_edge(4, 3, 9)
```

```
graph.add_edge(4, 5, 14)
```

```
graph.add_edge(5, 4, 14)
```

```
distance, previous = graph.dijkstra(0)
```

```
print("Vertex\tDistance\tPrevious")
```

```
for i in range(graph.vertices):
```

```
    print(f"{i}\t{distance[i]}\t{previous[i]}")
```

```
...
```

4. IMPORTANT QUESTIONS

Q1. (2 marks) What is Dijkstra's algorithm and what is its time complexity?

Answer: Dijkstra's algorithm is a graph search algorithm used to find the shortest path between nodes in a graph. Its time complexity is $O(E + V \log V)$.

Q2. (2 marks) What is the difference between single-source shortest path and single-destination shortest path in Dijkstra's algorithm?

Answer: Single-source shortest path finds the shortest path from a single source node to all other nodes in the graph, while single-destination shortest path finds the shortest path from all nodes to a single destination node.

Q3.